

# 实验八 软件需求规格说明（SRS）与动态建模

项目名称：BlogHub 博客社区平台

小组成员：王佳闻(evermore127)、王榕祺(ramble77)、杨小英(Qing1012)、田依然

完成日期：2026-05-15

## 第一部分：Petri 网理论学习总结

### 1.1 Petri 网基本概念

Petri 网是一种用于描述复杂系统中活动流程的图形化建模工具，由 Carl Adam Petri 于 1962 年在其博士论文中提出。相比于框图、逻辑树等传统图形化表示技术，Petri 网特别适合自然地表示系统各部分或活动之间的逻辑交互，如同步（Synchronization）、顺序性（Sequentiality）、并发（Concurrency）和冲突（Conflict）。

一个**有标的 Petri 网**（Marked Petri Net）定义为一个五元组：

$$PN = (P, T, I, O, Mo)$$

其中：

- $P = \{p_1, p_2, \dots, p_n\}$ ：有限库所（Place）集合，图形上用圆圈表示，代表系统的条件或状态
- $T = \{t_1, t_2, \dots, t_m\}$ ：有限变迁（Transition）集合，图形上用矩形/粗条表示，代表系统中的事件或活动
- $I$ ：输入函数，定义从库所到变迁的有向弧（输入弧），表示变迁发生所需的前提条件
- $O$ ：输出函数，定义从变迁到库所的有向弧（输出弧），表示变迁发生后产生的结果
- $Mo$ ：初始标识（Initial Marking），用库所中令牌（Token）的数量分布表示系统的初始状态

### 1.2 执行规则

Petri 网的动态行为通过令牌在库所间的移动来体现，遵循以下规则：

- 使能条件**：在标识  $M$  下，如果变迁  $t_k$  的每个输入库所  $p_i$  中都至少包含一个令牌，则称  $t_k$  在该标识下使能
- 点火规则**：使能的变迁通过**点火**执行——从每个输入库所移除一个令牌，向每个输出库所添加一个令牌

数学表达：从标识  $M$  点火变迁  $t_k$  得到新标识  $M'$ ：

- |   |                                                               |
|---|---------------------------------------------------------------|
| 1 | $M'(p_i) = M(p_i) - 1, \quad p_i \in I(t_k) \setminus O(t_k)$ |
| 2 | $M'(p_i) = M(p_i) + 1, \quad p_i \in O(t_k) \setminus I(t_k)$ |
| 3 | $M'(p_i) = M(p_i), \quad \text{其他}$                           |

### 1.3 Petri 网对系统建模的典型模式

论文详细阐述了 Petri 网在系统建模中的几种典型应用模式：

| 模式                       | 描述                 | 图示含义              |
|--------------------------|--------------------|-------------------|
| 并发 (Concurrency)         | 多个变迁同时使能，互不影响      | 如两个并联组件的故障过程      |
| 同步 (Synchronization)     | 多个并行活动完成后再继续执行     | 变迁需所有输入库所都有令牌才能点火 |
| 有限资源 (Limited Resources) | 资源数量有限，耗尽时阻塞活动     | 缓冲区的空位和占位构成库所不变量  |
| 顺序性 (Sequentiality)      | 生产者/消费者问题，活动严格按序进行 | 缓冲区令牌数为 1 时强制顺序执行 |
| 互斥 (Mutual Exclusion)    | 共享资源不能同时被多个进程访问    | 通过一个库所控制资源的访问权    |

## 1.4 Petri 网的性质分析

论文介绍了以下重要性质：

- 活性 (Liveness)**：如果从任何可达标识出发，都存在一个点火序列使某变迁最终使能，则该变迁是活的
- 安全性 (Safeness)**：若在任何可达标识中，库所中的令牌数不超过 1，则该库所是安全的
- 有界性 (Boundedness)**：若在任何可达标识中，库所中的令牌数不超过  $k$ ，则该库所是  $k$ -有界的
- 守恒性 (Conservation)**：若在任何可达标识中，令牌总数保持不变，则系统是严格守恒的

## 1.5 分析技术

- 可达树/可达图**：从初始标识出发，系统地枚举所有可能的标识和变迁点火序列，是分析 Petri 网动态行为的基本方法
- 矩阵分析**：通过关联矩阵  $D = D^+ - D^-$  和状态方程  $M' = M + X \cdot D$  进行数学分析，可以研究可达性、守恒性和库所不变量

## 1.6 扩展与随机 Petri 网

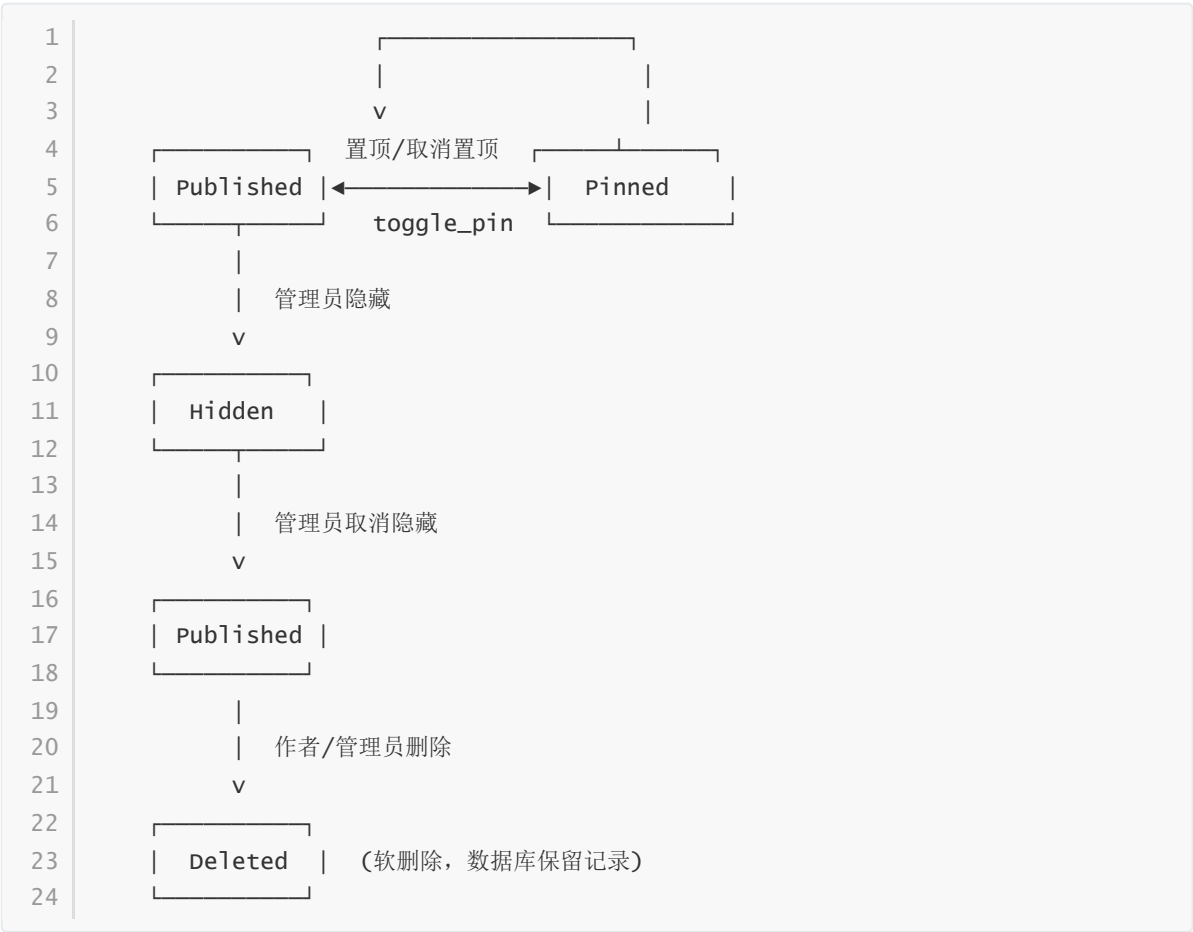
- 抑制弧 (Inhibitor Arc)**：当输入库所无令牌时使能变迁，用于实现优先级
- 优先级 (Priority Levels)**：为变迁分配优先级，高优先级先点火
- 随机 Petri 网 (SPN)**：为变迁关联指数分布的随机点火时间，可将可达图映射为连续时间马尔可夫链，用于系统的性能和可靠性定量分析

# 第二部分：动态建模工具应用

## 2.1 状态图 (State Diagram)

### 2.1.1 文章 (Post) 状态图

文章在 BlogHub 平台中经历从创建到删除的完整生命周期，涉及多个状态转换：



状态说明：

| 状态             | 含义       | 可见性         | 触发操作     |
|----------------|----------|-------------|----------|
| Published（已发布） | 文章正常发布   | 对所有人可见      | 创建文章     |
| Pinned（已置顶）    | 文章被置顶    | 对所有人可见，排序靠前 | 管理员置顶    |
| Hidden（已隐藏）    | 文章被管理员隐藏 | 仅管理员可见      | 管理员隐藏    |
| Deleted（已删除）   | 文章被软删除   | 不可见         | 作者或管理员删除 |

状态转换与权限：

1

2

3

4

5

6

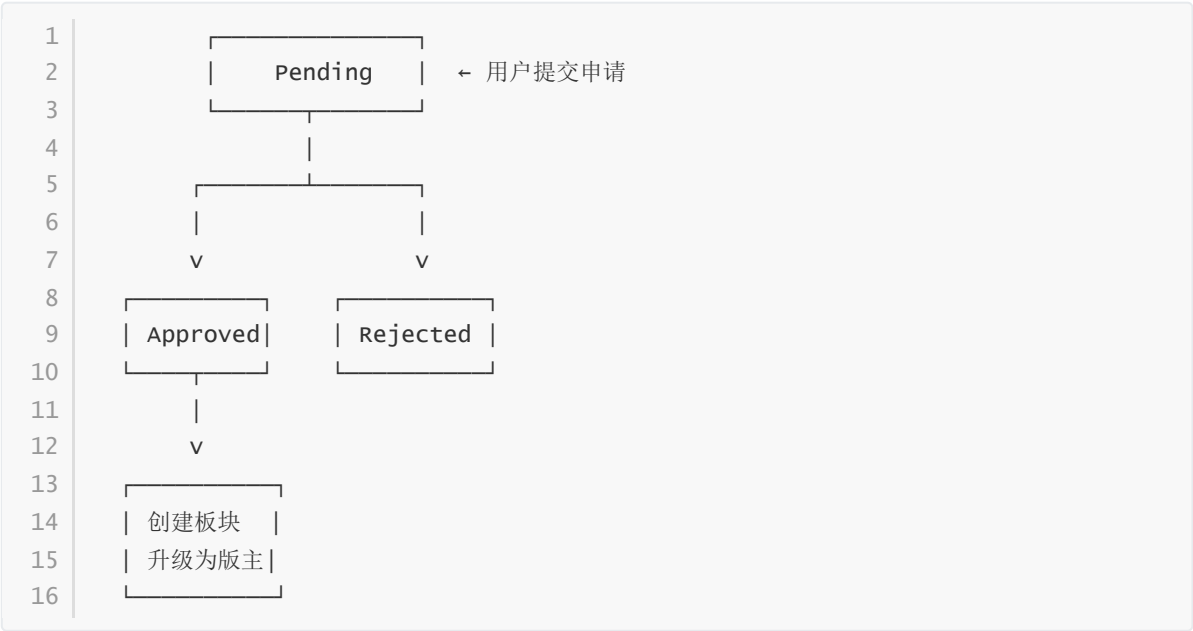
7

```
context Post inv:
  -- 只有已发布的文章才能被置顶
  is_pinned = true implies status = PostStatus::published

  -- 已删除的文章不可恢复
  status = PostStatus::deleted implies
    not (status@pre = PostStatus::published or status@pre =
      PostStatus::hidden)
```

### 2.1.2 板块申请 (SectionApplication) 状态图

用户申请创建板块需要经过管理员审批流程：



状态转换约束 (OCL)：

1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14

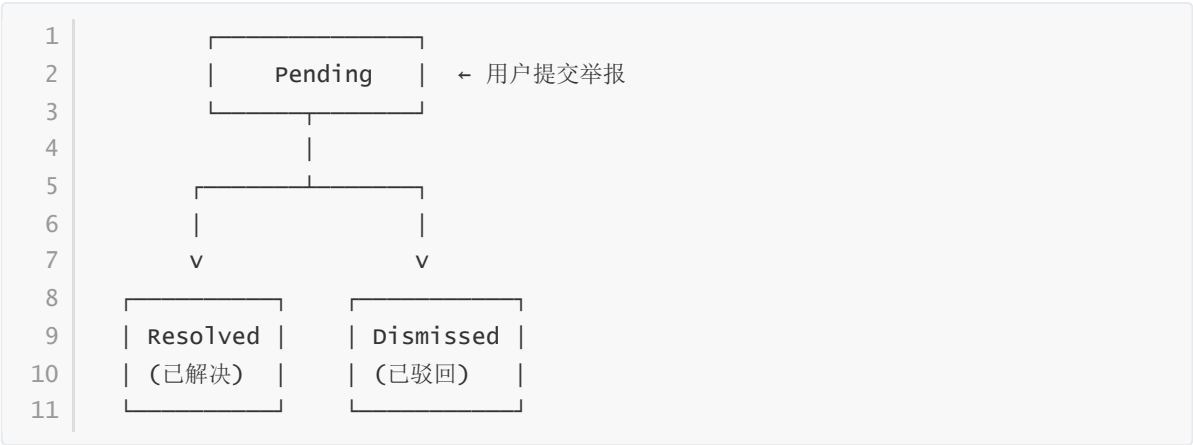
```
context SectionApplication inv:
-- 只有待审批的申请可以被批准或驳回
status = ApplicationStatus::approved or status =
ApplicationStatus::rejected
implies status@pre = ApplicationStatus::pending

-- 审批通过后必须关联审核人
status = ApplicationStatus::approved implies reviewer_id->notEmpty()

-- 一个用户只能有一个待审批的申请
context SectionApplication inv:
SectionApplication.allInstances()->select(
applicant_id = self.applicant_id and
status = ApplicationStatus::pending
)->size() <= 1
```

### 2.1.3 举报 (Report) 状态图

举报处理流程：



状态转换约束：

```
1 context Report inv:
2   -- 只有待处理的举报可以被处理
3   status = ReportStatus::resolved or status = ReportStatus::dismissed
4   implies status@pre = ReportStatus::pending
5
6   -- 已处理的举报必须有审核人
7   status <> ReportStatus::pending implies reviewer_id->notEmpty()
```

2.1.4 用户角色状态图



OCL 约束：

```
1 context User inv:
2   -- 角色枚举限定（当前无降级机制）
3   self.role = UserRole::user or
4   self.role = UserRole::moderator or
5   self.role = UserRole::admin
6
7   -- 版主不能直接成为管理员（需管理员手动操作）
8   context User::promoteToModerator() post:
9     self.role@pre = UserRole::user implies self.role = UserRole::moderator
```

2.2 Petri 网建模

2.2.1 文章生命周期 Petri 网模型

该 Petri 网模型描述了用户创建文章、互动（点赞/收藏）、管理员审核的完整流程。

库所 (Places) :

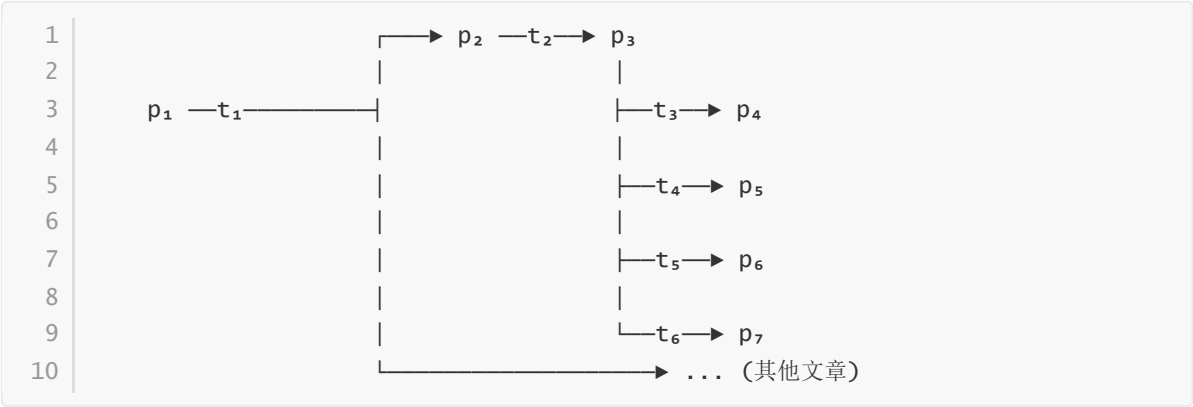
| 库所 | 含义         | 初始令牌数 |
|----|------------|-------|
| p1 | 用户处于活跃状态   | 1     |
| p2 | 文章已创建（待发布） | 0     |

| 库所             | 含义           | 初始令牌数 |
|----------------|--------------|-------|
| p <sub>3</sub> | 文章已发布        | 0     |
| p <sub>4</sub> | 文章已被阅读（浏览计数） | 0     |
| p <sub>5</sub> | 用户点赞/收藏操作    | 0     |
| p <sub>6</sub> | 文章被隐藏/置顶     | 0     |
| p <sub>7</sub> | 文章已删除（软删除）   | 0     |

变迁 (Transitions) :

| 变迁             | 含义       | 触发条件      |
|----------------|----------|-----------|
| t <sub>1</sub> | 用户创建文章   | 用户登录且填写内容 |
| t <sub>2</sub> | 发布文章     | 文章内容完整    |
| t <sub>3</sub> | 用户浏览文章   | 访问详情页     |
| t <sub>4</sub> | 用户点赞/收藏  | 点击互动按钮    |
| t <sub>5</sub> | 管理员隐藏/置顶 | 管理员权限     |
| t <sub>6</sub> | 删除文章     | 作者或管理员操作  |

Petri 网结构:



关联矩阵  $D = D^+ - D^-$ :

|   |       |       |       |       |       |       |       |       |    |
|---|-------|-------|-------|-------|-------|-------|-------|-------|----|
| 1 |       | $p_1$ | $p_2$ | $p_3$ | $p_4$ | $p_5$ | $p_6$ | $p_7$ |    |
| 2 | $t_1$ | [-1   | 1     | 0     | 0     | 0     | 0     | 0]    |    |
| 3 | $t_2$ | [     | 0     | -1    | 1     | 0     | 0     | 0     | 0] |
| 4 | $t_3$ | [     | 0     | 0     | 0     | 1     | 0     | 0     | 0] |
| 5 | $t_4$ | [     | 0     | 0     | 0     | 0     | 1     | 0     | 0] |
| 6 | $t_5$ | [     | 0     | 0     | -1    | 0     | 0     | 1     | 0] |
| 7 | $t_6$ | [     | 0     | 0     | -1    | 0     | 0     | 0     | 1] |

(浏览不消耗令牌)

可达图说明:

初始标识  $M_0 = (1, 0, 0, 0, 0, 0, 0, 0)$  表示用户空闲且无文章。点火  $t_1$  后进入  $M_1 = (0, 1, 0, 0, 0, 0, 0, 0)$ ，表示用户正在创作文章。再点火  $t_2$  进入  $M_2 = (0, 0, 1, 0, 0, 0, 0, 0)$  文章已发布，之后可以并发地发生浏览( $t_3$ )、互动( $t_4$ )、管理操作( $t_5/t_6$ )。

### 2.2.2 板块申请审批流程 Petri 网模型

该模型使用带抑制弧的 Petri 网描述板块申请的并发处理和资源互斥。

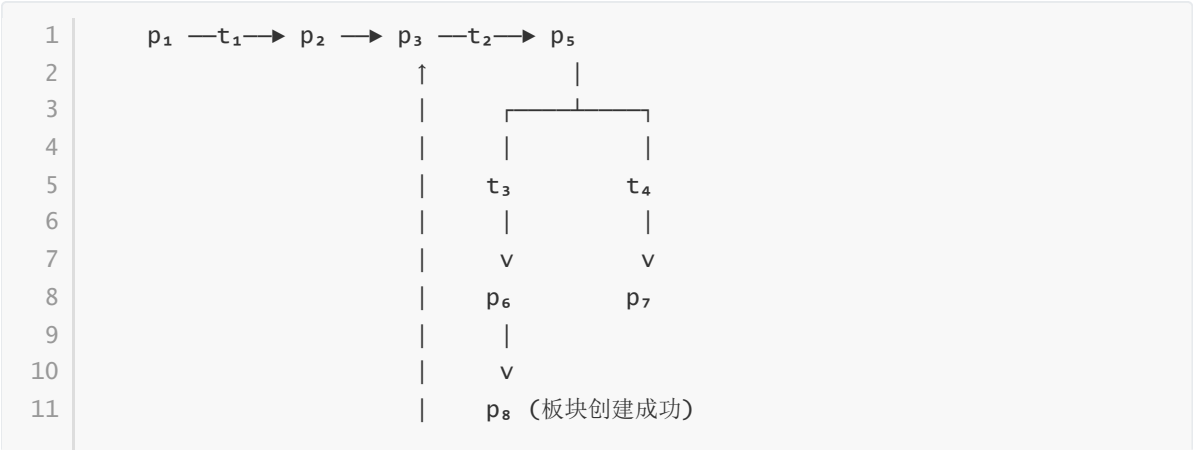
库所 (Places)：

| 库所    | 含义      | 初始令牌数   |
|-------|---------|---------|
| $p_1$ | 用户有资格申请 | N (用户数) |
| $p_2$ | 申请已提交   | 0       |
| $p_3$ | 申请待审核   | 0       |
| $p_4$ | 管理员空闲   | 1       |
| $p_5$ | 管理员正在审核 | 0       |
| $p_6$ | 申请已通过   | 0       |
| $p_7$ | 申请已驳回   | 0       |
| $p_8$ | 板块已创建   | 0       |

变迁 (Transitions)：

| 变迁    | 含义                |
|-------|-------------------|
| $t_1$ | 用户填写并提交申请         |
| $t_2$ | 管理员开始审核申请         |
| $t_3$ | 管理员批准申请           |
| $t_4$ | 管理员驳回申请           |
| $t_5$ | 创建板块并升级版主         |
| $t_6$ | 申请被挂起 (抑制弧, 防止并发) |

Petri 网结构：



|    |                                                           |
|----|-----------------------------------------------------------|
| 12 |                                                           |
| 13 | p <sub>4</sub> _____                                      |
| 14 |                                                           |
| 15 | 抑制弧: p <sub>5</sub> —○→ t <sub>2</sub> (当管理员正忙时, 其他审核被抑制) |

该模型反映了管理员作为"共享资源"的互斥访问——当管理员正在审核一个申请时，其他申请不能同时进入审核状态。

2.2.3 并发互动 Petri 网模型（点赞/收藏/评论）

库所 (Places) :

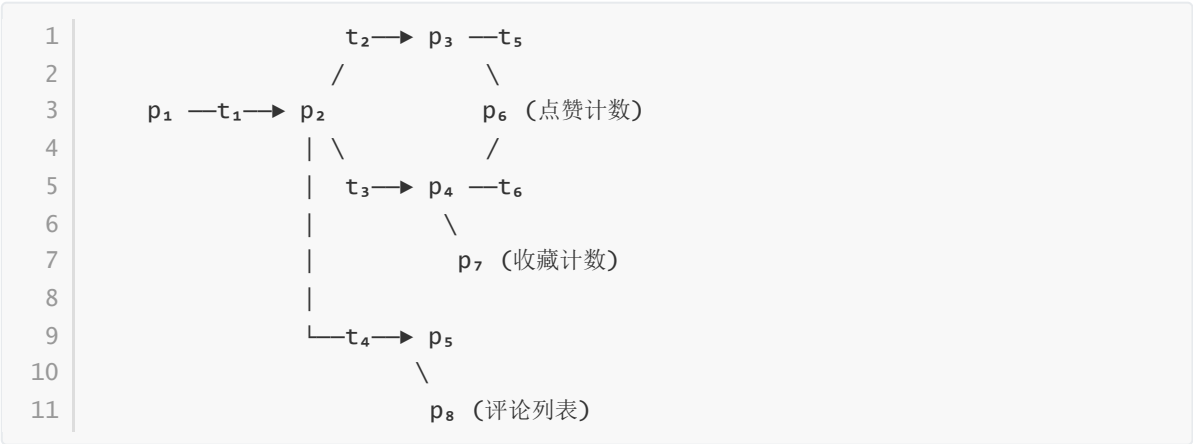
| 库所             | 含义     | 初始令牌数 |
|----------------|--------|-------|
| p <sub>1</sub> | 文章已发布  | 1     |
| p <sub>2</sub> | 用户阅读文章 | 0     |
| p <sub>3</sub> | 点赞操作就绪 | 0     |
| p <sub>4</sub> | 收藏操作就绪 | 0     |
| p <sub>5</sub> | 评论操作就绪 | 0     |
| p <sub>6</sub> | 点赞计数   | 0     |
| p <sub>7</sub> | 收藏计数   | 0     |
| p <sub>8</sub> | 评论列表   | 0     |

变迁 (Transitions) :

| 变迁             | 含义           |
|----------------|--------------|
| t <sub>1</sub> | 用户打开文章（开始阅读） |
| t <sub>2</sub> | 用户点击点赞       |
| t <sub>3</sub> | 用户点击收藏       |
| t <sub>4</sub> | 用户发表评论       |
| t <sub>5</sub> | 用户取消点赞       |
| t <sub>6</sub> | 用户取消收藏       |

Petri 网模型（并发模式）:

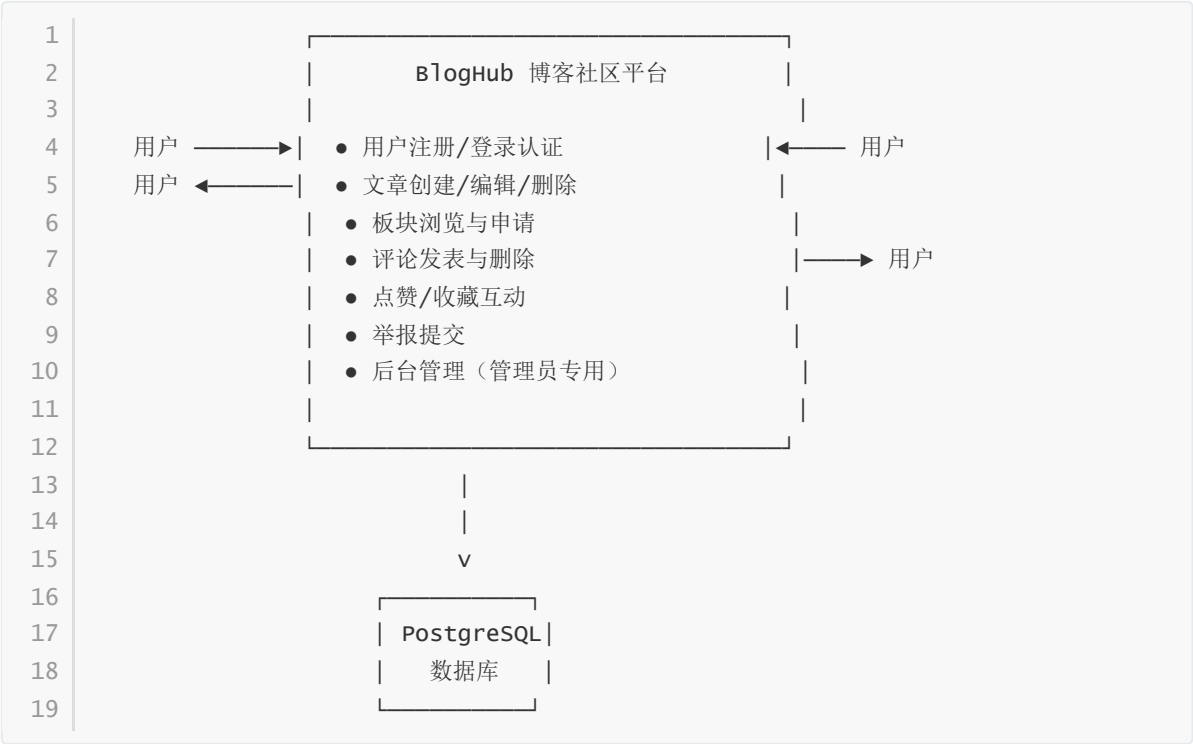




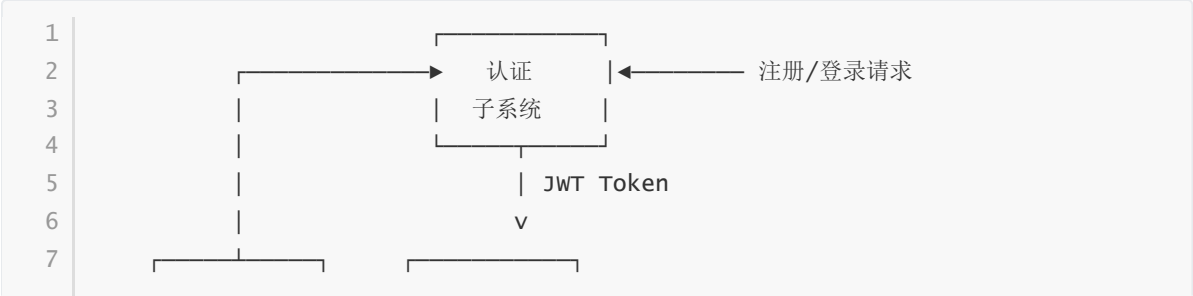
- 特性分析：**
- **并发：**从  $p_2$  出发， $t_2$ 、 $t_3$ 、 $t_4$  可以并发触发，体现了点赞、收藏、评论的独立执行特性
  - **互斥：**同一用户对同一篇文章的同一类型互动有唯一性约束（数据库层有  $\text{UNIQUE}(\text{user\_id}, \text{post\_id}, \text{type})$ ）
  - **有界性：** $p_6$ 、 $p_7$ 、 $p_8$  的令牌数受用户数量限制，系统是  $k$ -有界的

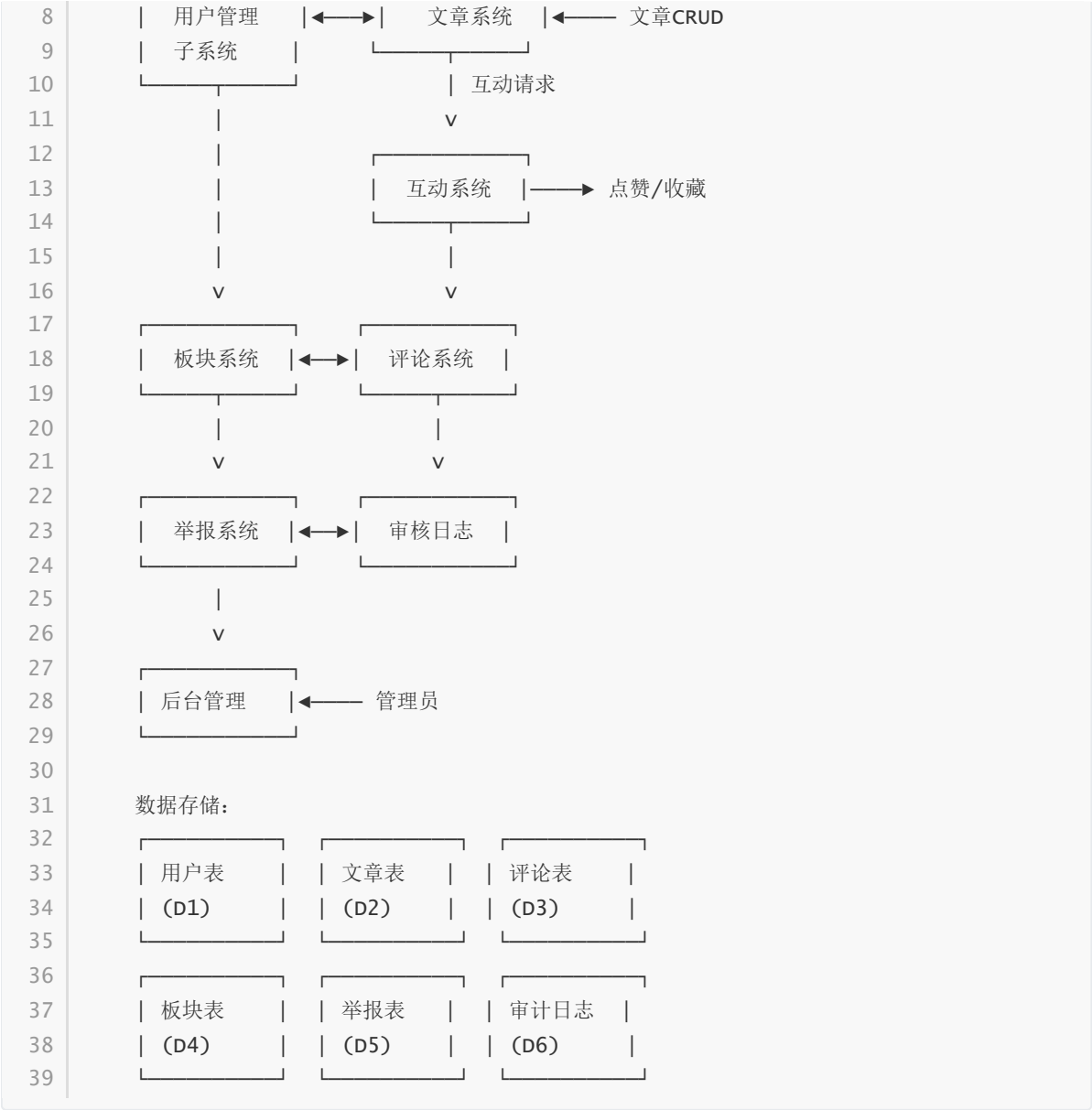
## 2.3 数据流图 (DFD)

### 2.3.1 上下文图 (Level-0)

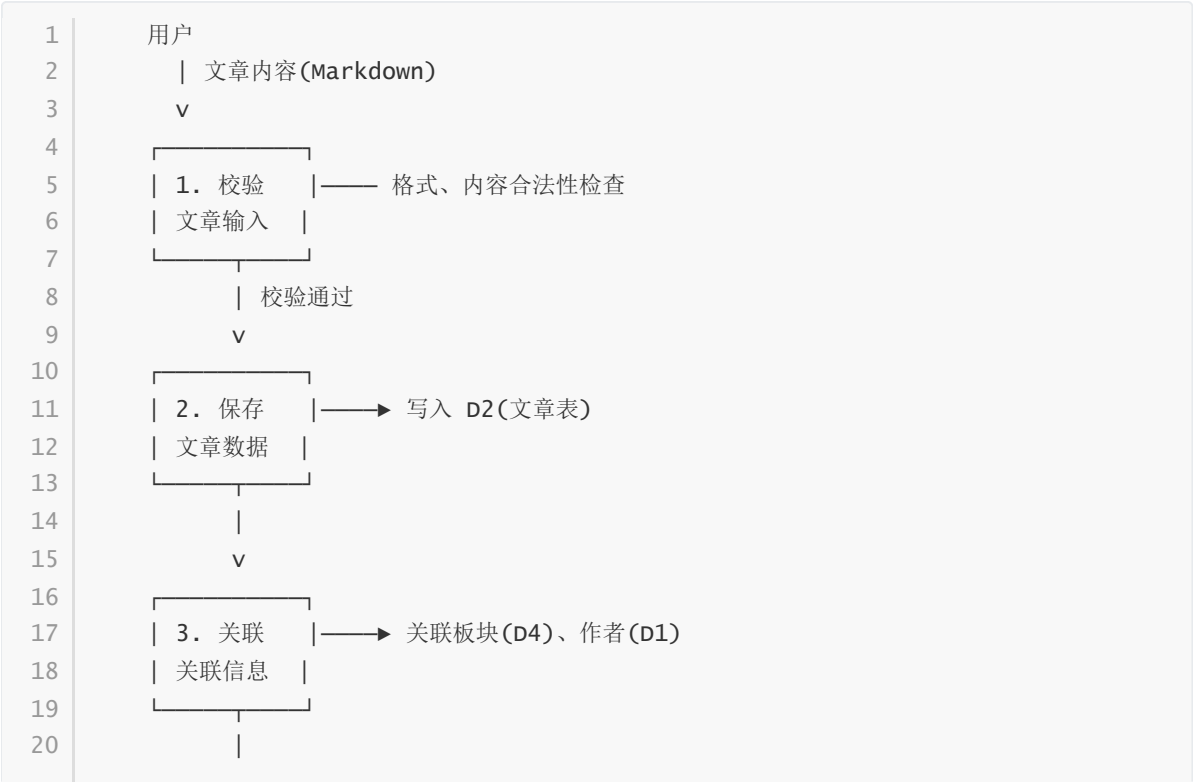


### 2.3.2 一层数据流图 (Level-1)





2.3.3 二层数据流图：文章发布流程



```

21      v
22
23      | 4. 返回 | —— 返回文章详情给用户
24      | 响应数据 |
25

```

## 2.4 OCL 逻辑约束

### 2.4.1 用户约束

```

1  -- 用户名长度约束
2  context User inv:
3      self.username->size() >= 3 and self.username->size() <= 50
4
5  -- 邮箱格式约束
6  context User inv:
7      self.email->includes('@') and self.email->includes('.')
8
9  -- 密码哈希不可为空
10 context User inv:
11     self.password_hash->notEmpty()
12
13 -- 版主/管理员约束：只有版主及以上角色才能管理板块
14 context User::canManageSection(section: Section): Boolean post:
15     result = (self.role = UserRole::moderator or self.role = UserRole::admin)

```

### 2.4.2 文章约束

```

1  -- 文章标题长度
2  context Post inv:
3      self.title->size() >= 1 and self.title->size() <= 200
4
5  -- 文章内容不可为空
6  context Post inv:
7      self.content->notEmpty()
8
9  -- 文章必须属于有效板块
10 context Post inv:
11     self.section.status = SectionStatus::active
12
13 -- 已删除的文章不可再互动
14 context Post inv:
15     self.status = PostStatus::deleted implies
16         self.interactions->isEmpty()
17
18 -- 置顶文章必须是已发布状态
19 context Post inv:
20     self.is_pinned = true implies self.status = PostStatus::published
21
22 -- 浏览计数的单调性（只增不减）
23 context Post::incrementViewCount() post:
24     self.view_count = self.view_count@pre + 1
25

```

```

26 -- 唯一互动约束
27 context PostInteraction inv:
28   PostInteraction.allInstances()->select(
29     user_id = self.user_id and
30     post_id = self.post_id and
31     type = self.type
32   )->size() = 1

```

### 2.4.3 评论约束

```

1  -- 评论内容不可为空
2  context Comment inv:
3    self.content->notEmpty()
4
5  -- 嵌套回复不能超过指定深度（最多 2 层嵌套）
6  context Comment inv:
7    Comment.allInstances()->select(
8      id = self.parent_id.parent_id
9    )->isEmpty()
10 -- 即 parent_id 不能再有 parent_id（实际实现中可支持任意嵌套，
11 -- 但业务上建议限制深度）
12
13 -- 删除评论的权限约束
14 context Comment::delete(user: User) pre:
15   user.id = self.author_id or
16   user.role = UserRole::moderator or
17   user.role = UserRole::admin

```

### 2.4.4 板块约束

```

1  -- 板块名称不可为空
2  context Section inv:
3    self.name->notEmpty() and self.name->size() <= 100
4
5  -- 板块嵌套深度约束（最多 2 级）
6  context Section inv:
7    self.parent_id->notEmpty() implies
8    self.parent.parent_id->isEmpty()
9
10 -- 板块申请的名称唯一性（同一层级下）
11 context SectionApplication inv:
12   Section.active.select(name = self.name)->isEmpty()
13
14 -- 申请审批流程约束
15 context SectionApplication::approve() post:
16   self.status = ApplicationStatus::approved and
17   self.reviewer_id->notEmpty() and
18   self.reviewed_at->notEmpty()
19
20 context SectionApplication::reject() post:
21   self.status = ApplicationStatus::rejected and
22   self.reviewer_id->notEmpty() and
23   self.reviewed_at->notEmpty()

```

## 2.4.5 举报约束

```
1  -- 举报原因不可为空
2  context Report inv:
3    self.reason->notEmpty()
4
5  -- 用户不能举报自己
6  context Report inv:
7    self.reporter_id <> self.target_type = ReportTargetType::user implies
8    self.target_id
9
10 -- 已处理的举报不可重复处理
11 context Report::resolve() pre:
12   self.status = ReportStatus::pending
13
14 context Report::dismiss() pre:
15   self.status = ReportStatus::pending
```

## 2.4.6 系统级约束

```
1  -- 审计日志必须记录操作人、操作类型、目标
2  context AuditLog inv:
3    self.operator_id->notEmpty() and
4    self.action->notEmpty() and
5    self.target_type->notEmpty() and
6    self.target_id->notEmpty()
7
8  -- 所有 API 统一响应格式
9  context APIResponse inv:
10   self.code = 0 implies self.msg = "ok"
11   self.code <> 0 implies self.msg->notEmpty()
12
13 -- 文件上传约束
14 context Upload inv:
15   self.file.size <= 5242880 and -- 5MB
16   self.file.extension->includesAny('jpg', 'jpeg', 'png', 'gif', 'webp')
```

---

# 第三部分：软件需求规格说明（SRS）

## 3.1 引言

### 3.1.1 目的

本文档旨在完整定义 BlogHub 博客社区平台的功能需求、非功能需求、系统约束和建模规约，为系统的开发、测试和验收提供依据。

### 3.1.2 范围

BlogHub 是一个基于 FastAPI + React + PostgreSQL 的全栈博客社区平台，支持多级板块分类、文章互动、评论系统和后台管理。

3.1.3 定义与缩写

| 术语        | 说明                                           |
|-----------|----------------------------------------------|
| JWT       | JSON Web Token，用于身份认证                        |
| SRS       | Software Requirements Specification，软件需求规格说明 |
| OCL       | Object Constraint Language，对象约束语言            |
| DFD       | Data Flow Diagram，数据流图                       |
| Petri Net | Petri 网，一种系统建模工具                             |

3.2 功能需求

3.2.1 用户系统

| 编号    | 功能     | 描述                                    | 优先级 |
|-------|--------|---------------------------------------|-----|
| FR-01 | 用户注册   | 用户提供用户名、邮箱、密码进行注册，密码使用 bcrypt 哈希存储    | 高   |
| FR-02 | 用户登录   | 用户使用邮箱和密码登录，返回 JWT 令牌（7 天有效）          | 高   |
| FR-03 | 个人信息查看 | 查看个人资料（用户名、邮箱、头像、个人简介、角色）             | 高   |
| FR-04 | 个人信息编辑 | 编辑用户名、个人简介、修改密码（需验证当前密码）              | 高   |
| FR-05 | 头像上传   | 上传头像（支持 jpg/jpeg/png/gif/webp，最大 5MB） | 中   |
| FR-06 | 用户资料查看 | 查看其他用户的公开资料                           | 中   |

3.2.2 板块系统

| 编号    | 功能     | 描述                               | 优先级 |
|-------|--------|----------------------------------|-----|
| FR-07 | 板块树查看  | 查看按层级组织的板块树结构                    | 高   |
| FR-08 | 板块详情查看 | 查看单个板块的详细信息                      | 高   |
| FR-09 | 板块申请   | 用户提交创建板块的申请（一级/二级板块）             | 高   |
| FR-10 | 板块申请审批 | 管理员审批/驳回板块申请（通过后自动创建板块，申请人升级为版主） | 高   |

3.2.3 文章系统

| 编号    | 功能      | 描述                                | 优先级 |
|-------|---------|-----------------------------------|-----|
| FR-11 | 文章列表    | 按板块、搜索关键词、排序方式（时间/点赞/收藏）分页查看文章列表  | 高   |
| FR-12 | 文章详情    | 查看文章完整内容（包含作者信息、互动计数、浏览计数自增）      | 高   |
| FR-13 | 创建文章    | 用户创建 Markdown 文章，指定所属板块、标题、内容、封面图 | 高   |
| FR-14 | 编辑文章    | 作者或版主/管理员编辑文章                     | 高   |
| FR-15 | 删除文章    | 作者或版主/管理员软删除文章                    | 高   |
| FR-16 | 点赞/取消点赞 | 用户对文章点赞或取消点赞                      | 中   |
| FR-17 | 收藏/取消收藏 | 用户对文章收藏或取消收藏                      | 中   |

3.2.4 评论系统

| 编号    | 功能   | 描述                    | 优先级 |
|-------|------|-----------------------|-----|
| FR-18 | 查看评论 | 查看文章的评论列表（嵌套结构，按时间倒序） | 高   |
| FR-19 | 发表评论 | 用户对文章发表评论（支持嵌套回复）     | 高   |
| FR-20 | 删除评论 | 评论作者或版主/管理员删除评论       | 中   |

3.2.5 举报系统

| 编号    | 功能   | 描述                       | 优先级 |
|-------|------|--------------------------|-----|
| FR-21 | 提交举报 | 用户对文章、评论或其他用户提交举报（含举报原因） | 中   |
| FR-22 | 举报处理 | 管理员查看待处理举报，选择已解决或驳回      | 中   |

3.2.6 管理后台

| 编号    | 功能     | 描述              | 优先级 |
|-------|--------|-----------------|-----|
| FR-23 | 举报管理   | 管理员查看并处理所有待处理举报 | 高   |
| FR-24 | 板块申请管理 | 管理员查看并审批/驳回板块申请 | 高   |

| 编号    | 功能     | 描述                            | 优先级 |
|-------|--------|-------------------------------|-----|
| FR-25 | 文章管理   | 管理员查看所有文章，进行置顶/取消置顶、隐藏/取消隐藏操作 | 高   |
| FR-26 | 审计日志查看 | 管理员查看最近 100 条审计日志             | 中   |

### 3.2.7 文件上传

| 编号    | 功能   | 描述                       | 优先级 |
|-------|------|--------------------------|-----|
| FR-27 | 图片上传 | 上传图片（格式和大小校验），UUID 重命名存储 | 中   |

## 3.3 非功能需求

| 编号     | 需求    | 描述                               |
|--------|-------|----------------------------------|
| NFR-01 | 响应时间  | API 响应时间在正常负载下不超过 500ms          |
| NFR-02 | 并发支持  | 系统应支持至少 100 并发用户访问               |
| NFR-03 | 安全性   | 密码使用 bcrypt 哈希存储；JWT 认证；API 权限校验 |
| NFR-04 | 可用性   | 系统应保证 99.9% 的可用时间                |
| NFR-05 | 可维护性  | 使用容器化部署，Alembic 管理数据库迁移          |
| NFR-06 | 数据一致性 | 评论、互动等数据应保持强一致性（数据库层约束）          |

## 3.4 系统约束

| 编号   | 约束     | 说明                                                      |
|------|--------|---------------------------------------------------------|
| C-01 | 认证方式   | JWT 令牌认证，7 天有效期                                         |
| C-02 | 数据库    | PostgreSQL 16，异步 SQLAlchemy 2.0                         |
| C-03 | API 格式 | 统一格式 <code>{"code": 0, "data": ..., "msg": "ok"}</code> |
| C-04 | 部署方式   | Docker Compose 三容器编排（PostgreSQL + Backend + Frontend）   |

## 3.5 数据库安全策略

### 3.5.1 用户密码

```
1 -- 密码使用 bcrypt 哈希存储，绝不存储明文
2 -- passlib.context.CryptContext(schemes=["bcrypt"])
3 -- 哈希长度：60 字符
4 COLUMN password_hash VARCHAR(255) NOT NULL
```



3.5.2 数据完整性约束

| 表                 | 约束类型                                              | 说明               |
|-------------------|---------------------------------------------------|------------------|
| users             | UNIQUE(username, email)                           | 用户名和邮箱全局唯一       |
| posts             | FK → users(id), FK → sections(id)                 | 文章必须关联有效作者和板块    |
| comments          | FK → posts(id), FK → users(id), FK → comments(id) | 自引用外键实现嵌套评论      |
| post_interactions | UNIQUE(user_id, post_id, type)                    | 同一用户对同一文章的互动类型唯一 |
| sections          | FK → sections(id)                                 | 自引用父外键实现嵌套       |

3.6 接口规范

所有 API 端点遵循以下统一规范：

```
1  请求：
2    Method: GET/POST/PUT/DELETE
3    Content-Type: application/json （文件上传使用 multipart/form-data）
4    Authorization: Bearer <JWT Token> （需要认证的接口）
5
6  响应：
7    {
8      "code": 0,           // 0 表示成功，非 0 表示错误
9      "data": {...},      // 响应数据
10     "msg": "ok"          // 错误时的错误信息
11   }
```

3.7 角色权限矩阵

| 功能      | 未登录 | User | Moderator | Admin |
|---------|-----|------|-----------|-------|
| 查看文章列表  | ✓   | ✓    | ✓         | ✓     |
| 查看文章详情  | ✓   | ✓    | ✓         | ✓     |
| 注册/登录   | ✓   | -    | -         | -     |
| 创建/编辑文章 | -   | ✓    | ✓         | ✓     |
| 删除文章    | -   | 仅自己的 | 板块内       | 全部    |
| 发表评论    | -   | ✓    | ✓         | ✓     |
| 删除评论    | -   | 仅自己的 | 板块内       | 全部    |
| 提交举报    | -   | ✓    | ✓         | ✓     |
| 申请板块    | -   | ✓    | ✓         | ✓     |
| 审批板块申请  | -   | -    | -         | ✓     |

| 功能      | 未登录 | User | Moderator | Admin |
|---------|-----|------|-----------|-------|
| 置顶/隐藏文章 | -   | -    | -         | ✓     |
| 管理举报    | -   | -    | -         | ✓     |
| 查看审计日志  | -   | -    | -         | ✓     |

## 第五部分：小组分工

| 成员  | 分工                                                           |
|-----|--------------------------------------------------------------|
| 王佳闻 | Petri 网理论学习与总结；文章/板块/评论系统的 Petri 网建模；状态图设计；OCL 约束撰写；参与实验报告撰写 |
| 王榕祺 | 数据流图设计（上下文图、一层/二层 DFD）；DFD 说明文档；参与 SRS 功能需求部分的完善             |
| 杨小英 | SRS 非功能需求、接口规范、权限矩阵撰写；需求验证与一致性检查；参与 Petri 网建模讨论              |
| 田依然 | OCL 逻辑约束撰写与验证；数据库安全策略设计；参与状态图设计和实验报告的结构编排                    |

## 参考资料

1. Andrea Bobbio. *SYSTEM MODELLING WITH PETRI NETS*. Istituto Elettrotecnico Nazionale Galileo Ferraris, 1990.
2. Peterson, J.L. *Petri Net Theory and the Modeling of Systems*. Prentice Hall, 1981.
3. BlogHub 项目源码：FastAPI + React + PostgreSQL 全栈博客社区平台
4. OMG. *Object Constraint Language Specification*, Version 2.4, 2014.